[H01] Life Before Frameworks

initDatabase

```
import http from "node:http";
import url from "node:url";
import Database from "better-sqlite3";
import path from "node:path";

// Database setup
const dbPath = path.join(process.cwd(), "users.db");
let db: Database.Database;

// Initialize database
const initDatabase = () => {
  if (!db) {
    db = new Database(dbPath);

    db.exec(`
      CREATE TABLE IF NOT EXISTS users (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT NOT NULL,
        age INTEGER NOT NULL,
        isAdmin INTEGER NOT NULL DEFAULT 0,
        createdAt TEXT NOT NULL,
        updatedAt TEXT NOT NULL
      )
    `);// make the table using sql

    db.exec(`CREATE INDEX IF NOT EXISTS idx_users_name ON users (name)`);
    db.exec(`CREATE INDEX IF NOT EXISTS idx_users_isAdmin ON users (isAdmin)`);// make index
  }
};
```

database inisialisation

Helper Functions

send json

Standardizes how the server responds

```
const sendJson = (res: http.ServerResponse, status: number, data: unknown) => {
  res.writeHead(status, { "Content-Type": "application/json" });
  res.end(JSON.stringify(data));
}; // send json response with status
```

row to user

Converts database so easier to process.

```
const rowToUser = (row: any): User => {
  return {
    id: row.id,
    name: row.name,
    age: row.age,
    isAdmin: row.isAdmin === 1,
    createdAt: row.createdAt,
    updatedAt: row.updatedAt,
  }; // since SQLite saves boolean as 1/0 not true or false, it converts it back
     // to true/false, and the other fields are written again to follow the table
};
```

parse id

Converts path string into numbers and validates it.

```
const parseId = (text: string): number | null => {
  const id = Number(text);
  if (text === "" || !Number.isInteger(id)) {
    return null;
  }
  return id;
}; // basically change string to number so it can be processed,
   // and reject text or float
```

validate the user

Checks user input so it doesnt exceed the amount requested

```
const validateUser = (data: any): string | null => {
  if (!data || typeof data !== "object" || Array.isArray(data) || Object.keys(data).length
=== 0) {
    return "Request body is required";
  } // make sure it is not empty
  if (typeof data.name !== "string" || data.name.trim() === "") {
    return "Name is required and must be a non-empty string";
  } // make sure name is a string and not empty
  if (data.name.trim().length > 100) {
    return "Name must be 100 characters or less";
  } // make sure the length is 100 characters or less
  if (data.age === undefined) {
    return "Age is required";
  } // make sure it is not missing
  if (typeof data.age !== "number" || !Number.isInteger(data.age) || data.age < 0 || data.age
> 150) {
    return "Age must be a valid integer between 0 and 150";
  } // must be a number, an integer, and not below 0 or above 150,
    // so it doesn't have a weird age
  if (data.isAdmin !== undefined && typeof data.isAdmin !== "boolean") {
    return "isAdmin must be a boolean value";
  } // make sure the value is boolean for validation
  return null;
};
```

read body to json

Collects data streams from HTTP requests. Parses text into JSON

```
// this reads the request and converts it to json
const readBody = (
  req: http.IncomingMessage,
  callback: (data: any, hasJsonError: boolean) => void
) => {
  let body = "";
  req.on("data", (chunk) => {
    body += chunk.toString();
  });
  req.on("end", () => {
    if (body === "") {
      callback(undefined, false);
      return;
    }
    let data: any;
    try {
      data = JSON.parse(body);
```

```
    } catch (error) {
      callback(undefined, true);
      return;
    }
    callback(data, false);
  });
};
```

create server

```
export const createMyServer = (): http.Server => {
  // Initialize database when server is created
  initDatabase();
  // CORS headers so the browser does not block GET, POST, PUT, DELETE requests
  const server = http.createServer((req, res) => {
    res.setHeader("Access-Control-Allow-Origin", "*");
    res.setHeader("Access-Control-Allow-Methods", "GET, POST, PUT, DELETE");
    res.setHeader("Access-Control-Allow-Headers", "Content-Type");
    // Preflight request
    if (req.method === "OPTIONS") {
      res.writeHead(200);
      res.end();
      return;
    }

    const parsedUrl = url.parse(req.url!, true);
    const pathName = parsedUrl.pathname;
    const method = req.method;
```

GET

Retrieves all users or a single user by ID from the database and returns them as JSON.

```
    // GET /users
    if (method === "GET" && pathName === "/users") {
      const rows = db.prepare("SELECT * FROM users ORDER BY id ASC").all();
      const users = rows.map(rowToUser);
      sendJson(res, 200, users);
      return;
    }

    // GET /users/:id, extract the id after /users/
    if (method === "GET" && pathName?.startsWith("/users/")) {
      const id = parseId(pathName.split("/")[2]);
      if (id === null) {
        sendJson(res, 400, { error: "Invalid user ID" });
        return;
      }
      // after getting the id, check the database for that id, if not found then error
      const row = db.prepare("SELECT * FROM users WHERE id = ?").get(id);
      if (!row) {
        sendJson(res, 404, { error: "User not found" });
        return;
      }
      // if all is done, send the data
      sendJson(res, 200, rowToUser(row));
      return;
    }
```

POST

Validates request data, inserts a new user record into SQLite, and returns the created user.

```
    // POST /users, create a user
    if (method === "POST" && pathName === "/users") {
      readBody(req, (data, hasJsonError) => {
        if (hasJsonError) {
          sendJson(res, 400, { error: "Invalid JSON" });
          return;
        }
        // give the error
        const errorMessage = validateUser(data);
        if (errorMessage) {
          sendJson(res, 400, { error: "Validation failed", details: errorMessage });
          return;
        }
        // prepare the data so it follows the table
        const name = data.name.trim();
        const age = data.age;
        const isAdmin = data.isAdmin === true ? 1 : 0;
        const now = new Date().toISOString();
        // put the data into the database
        const result = db
          .prepare("INSERT INTO users (name, age, isAdmin, createdAt, updatedAt) VALUES
(?, ?, ?, ?, ?)")
          .run(name, age, isAdmin, now, now);
        // check the data by select
        const newRow = db.prepare("SELECT * FROM users WHERE id
= ?").get(result.lastInsertRowid);
        sendJson(res, 201, rowToUser(newRow));
      });
      return;
    }
```

PUT

Checks if a user exists, validates incoming updates and modifies the user record in the database.

```
    // PUT /users/:id, update a user
    if (method === "PUT" && pathName?.startsWith("/users/")) {
      // this gets the id
      const id = parseId(pathName.split("/")[2]);
      if (id === null) {
        sendJson(res, 400, { error: "Invalid user ID" });
        return;
      }

      readBody(req, (data, hasJsonError) => {
        // read the data, if not valid throw error
        if (hasJsonError) {
          sendJson(res, 400, { error: "Invalid JSON" });
          return;
        }

        const errorMessage = validateUser(data); // give error message
        if (errorMessage) {
          sendJson(res, 400, { error: "Validation failed", details: errorMessage });
          return;
        }
        // check if it exists
```

```
        const existingRow = db.prepare("SELECT * FROM users WHERE id = ?").get(id);
        if (!existingRow) {
          sendJson(res, 404, { error: "User not found" });
          return;
        }
        // prepare the data so it follows the table
        const name = data.name.trim();
        const age = data.age;
        const isAdmin = data.isAdmin === true ? 1 : 0;
        const now = new Date().toISOString();
        // update the table row
        db.prepare("UPDATE users SET name = ?, age = ?, isAdmin = ?, updatedAt = ? WHERE id
= ?")
          .run(name, age, isAdmin, now, id);
        // select the id to show the update and send it
        const updatedRow = db.prepare("SELECT * FROM users WHERE id = ?").get(id);
        sendJson(res, 200, rowToUser(updatedRow));
      });
      return;
    }
```

DELETE

Removes a specified user record from the database and returns a confirmation message with the deleted user data.

```
    // DELETE /users/:id, check the id again
    if (method === "DELETE" && pathName?.startsWith("/users/")) {
      const id = parseId(pathName.split("/")[2]);
      if (id === null) {
        sendJson(res, 400, { error: "Invalid user ID" });
        return;
      }
      // check if the row exists, if not throw error
      const row = db.prepare("SELECT * FROM users WHERE id = ?").get(id);
      if (!row) {
        sendJson(res, 404, { error: "User not found" });
        return;
      }
      // delete the user based on id
      db.prepare("DELETE FROM users WHERE id = ?").run(id);
      sendJson(res, 200, { message: "User deleted", user: rowToUser(row) });
      return;
    }

    // nothing matched
    sendJson(res, 404, { error: "Not found" });
  });

  return server;
};
```